Assignment - 05

Querying – Aggregate Operations

On sales database :

1. Compute total sales of a given item (say item code=1103).

   Relational Algebra:

   $$\mathcal{F}_{SUM(qty*rate) \rightarrow total_sales}(\sigma_{itemcode=1103}(invoicedetails))$$

   Query:

```
----- SALES DATABASE -----
SET search_path TO sales, public;

--1. Compute total sales of a given item (say item code=1103).
SELECT SUM(id.qty * id.rate) AS total_sales
FROM invoicedetails AS id
WHERE id.itemcode = 1103;
```

   Output:

Data Output | Messages | Notifications

	total_sales bigint
1	8125000

2. What is sale for a given date? [Total sale amount for a give date; it can be computed by summing (qty*rate) from InvoiceDetails of invoices on given date].

   Relational Algebra:

   $$\mathcal{F}_{SUM(qty*rate) \rightarrow total_sale}(invoicedetails \bowtie \sigma_{invdate='2008-08-15'}(invoice))$$

   Query:

```
--2. What is sale for a given date? [Total sale amount for a give date; it can be computed by summing (qty*rate) from InvoiceDetails o
SELECT SUM(id.qty * id.rate) AS total_sale
FROM invoice AS i
JOIN invoicedetails AS id
    ON i.invno = id.invno
WHERE i.invdate = DATE '2008-08-15';
```

   Output:

Data Output | Messages | Notifications

	total_sale bigint
1	[null]

3. List item-codes of top 3 most sold item based on quantity.

Assignment - 05

Relational Algebra:

$\pi_{itemcode}(\tau_{total_quantity\ DESC}(\gamma_{itemcode,SUM(qty)->total_quantity}(invoicedetails)))\ [Limit\ 3]$

Query:

```
--3. List item-codes of top 3 most sold item based on quantity.
SELECT itemcode,
       SUM(qty) AS total_quantity
FROM invoicedetails
GROUP BY itemcode
ORDER BY total_quantity DESC
LIMIT 3;
```

Output:

Data Output | Messages | Notifications

	itemcode smallint	total_quantity bigint
1	1103	350
2	1102	175
3	1104	140

4. List (item codes, item name, category) of top 3 most sold item based on quantity.

Relational Algebra:

$\pi_{code,name,category}(\tau_{total_quantity\ DESC}(\gamma_{code,name,category,SUM(qty)->total_quantity}(items \bowtie_{code=itemcode} invoicedetails)))\ [Limit\ 3]$

Query:

```
--4. List (item codes, item name, category) of top 3 most sold item based on quantity.
SELECT it.code,
       it.name,
       it.category,
       SUM(id.qty) AS total_quantity
FROM items AS it
JOIN invoicedetails AS id
    ON it.code = id.itemcode
GROUP BY it.code, it.name, it.category
ORDER BY total_quantity DESC
LIMIT 3;
```

Output:

Assignment - 05

Data Output | Messages | Notifications

	code [PK] smallint	name character varying (20)	category integer	total_quantity bigint
1	1103	Key Board	3	350
2	1102	Pen Drive	2	175
3	1104	Mouse	4	140

5. Most valuable customer (customer id) in terms of purchase values. Customer that sums of maximum sale amount.

Relational Algebra:

$$\pi_{customerid}(\tau_{total_purchase\ DESC}(\gamma_{customerid,SUM(qty*rate)->total_purchase}(invoice \bowtie invoicedetails))) \ [Limit\ 1]$$

Query:

```
--5. Most valuable customer (customer id) in terms of purchase values. Customer that sums of maximum sale amount.
SELECT i.customerid
FROM invoice AS i
JOIN invoicedetails AS id
    ON i.invno = id.invno
GROUP BY i.customerid
ORDER BY SUM(id.qty * id.rate) DESC
LIMIT 1;
```

Output:

Data Output | Messages | Notifications

	customerid character (3)
1	C01

6. Most valuable customer (customer id) in terms profit to the company. Assume that profit on an item sale can be computed by formula: [Rate from invoicedetails relation] – [Average Purchase Price].

Relational Algebra:

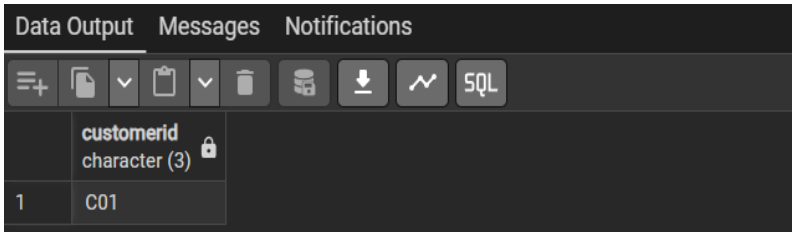
$$\pi_{customerid}(\tau_{total_profit\ DESC}(\gamma_{customerid,SUM(qty*(rate-averagepurchaseprice))->total_profit}(invoice \bowtie invoicedetails \bowtie_{itemcode=code} items))) \ [Limit\ 1]$$

Query:

```
--6. Most valuable customer (customer id) in terms profit to the company. Assume that profit on an item sale can be computed by formul
SELECT i.customerid
FROM invoice AS i
JOIN invoicedetails AS id
    ON i.invno = id.invno
JOIN items AS it
    ON id.itemcode = it.code
GROUP BY i.customerid
ORDER BY SUM(id.qty * (id.rate - it.averagepurchaseprice)) DESC
LIMIT 1;
```

Output:

Assignment - 05



7. Top selling item (in terms of numbers) for a given year.

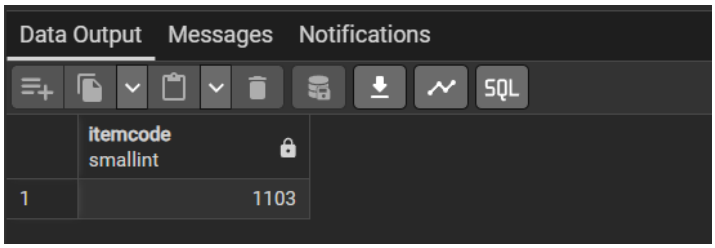
Relational Algebra:

$\pi_{itemcode}(\tau_{total_qty\ DESC}(\gamma_{itemcode,SUM(qty)->total_qty}(\sigma_{EXTRACT(YEAR\ FROM\ invdate)=2011}(invoice \bowtie invoicedetails))))\ [Limit\ 1]$

Query:

```
--7. Top selling item (in terms of numbers) for a given year.
SELECT id.itemcode
FROM invoice AS i
JOIN invoicedetails AS id ON i.invno = id.invno
WHERE EXTRACT(YEAR FROM i.invdate) = 2011
GROUP BY id.itemcode
ORDER BY SUM(id.qty) DESC
LIMIT 1;
```

Output:



On da-acad database :

8. Retrieve ID of faculties who took more than one courses in a semester (for all semester in the database).

Relational Algebra:

$\pi_{instructorid}(\sigma_{course_count>1}(\gamma_{instructorid,semester,acadyear,COUNT(courseno)->course_count}(offers)))$

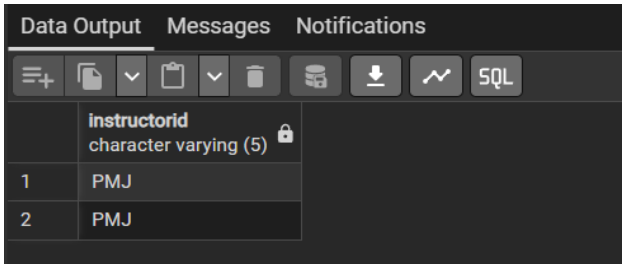
Query:

```
----- ACAD DATABASE -----
SET search_path TO acad, public;

-- 8. Retrieve ID of faculties who took more than one courses in a semester (for all semester in the database).
SELECT instructorid
FROM offers
GROUP BY instructorid, semester, acadyear
HAVING COUNT(courseno) > 1;
```

Output:

Assignment - 05



Data Output | Messages | Notifications

	instructorid character varying (5)
1	PMJ
2	PMJ

9. List total course count (instructor-name, course-count) for all instructors. Include the faculty name even if course count is zero.

Relational Algebra:

$$\gamma_{instructorid,instructorname,COUNT(courseno)->course_count}(instructor \bowtie_{instructor.instructorid=offers.instructorid} offers)$$

Query:

```
-- 9. List total course count (instructor-name, course-count) for all instructors. Include the faculty name even if course count is ze
SELECT i.instructorname, COUNT(o.courseno) AS course_count
FROM instructor AS i
LEFT JOIN offers AS o ON i.instructorid = o.instructorid
GROUP BY i.instructorid, i.instructorname;
```

Output:

Data Output | Messages | Notifications

	instructorname character varying (30)	course_count bigint
1	Mehul Raval	4
2	To be Decided	0
3	Manish Khare	0
4	V Sunita	0
5	Vijay Kumar Chakka	0
6	Sanjay Srivastava	3
7	M V Joshi	2
8	P M Jat	8
9	Saumin Maitra	1
10	Pankaj Kumar	3
11	Manish Gupta	0

10. Retrieve all students (StudentID, Name, TotalCreditTaken) for B.Tech. (CS) (progid='02') batch 2007 in Autumn'2008.

Relational Algebra:

$$\gamma_{studentid,name,SUM(credit)->total_credit_taken}(\sigma_{progid='02' \wedge batch=2007 \wedge semester='Autumn' \wedge acadyear=2008}(student \bowtie registers \bowtie course))$$

Assignment - 05

Query:

```sql
-- 10. Retrieve all students (StudentID, Name, TotalCreditTaken) for B.Tech. (CS) (progid='02') batch 2007 in Autumn'2008.
SELECT s.studentid, s.name, SUM(c.credit) AS total_credit_taken
FROM student AS s
JOIN registers AS r ON s.studentid = r.studentid
JOIN course AS c ON r.courseno = c.courseno
WHERE s.progid = '02'
  AND s.batch = 2007
  AND r.semester = 'Autumn'
  AND r.acadyear = 2008
GROUP BY s.studentid, s.name;
```

Output:

Data Output | Messages | Notifications

	studentid [PK] character varying (9)	name character varying (30)	total_credit_taken numeric
1	200702001	Sumit Sharma	4.0
2	200702002	Amit Kumar	4.0
3	200702003	Shobha Kiran	4.0
4	200702004	Raj Gupta	4.0
5	200702005	Amit Tiwari	4.0

11. Retrieve all students (Id and name) who got more than two F grades in Autumn'2008.

Relational Algebra:

$$\pi_{studentid,name}(\sigma_{f_count>2}(\gamma_{studentid,name,COUNT(*)->f_count}(\sigma_{grade='F'\wedge semester='Autumn'\wedge acadyear=2008}(student \bowtie registers))))$$

Query:

```sql
-- 11. Retrieve all students (Id and name) who got more than two F grades in Autumn'2008.
SELECT s.studentid, s.name
FROM student AS s
JOIN registers AS res ON s.studentid = res.studentid
WHERE res.grade = 'F'
  AND res.semester = 'Autumn'
  AND res.acadyear = 2008
GROUP BY s.studentid, s.name
HAVING COUNT(*) > 2;
```

Output:

Data Output | Messages | Notifications

	studentid [PK] character varying (9)	name character varying (30)